

Lab Assignment 8

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



Submitted By:

Mridul Sharma

Roll No. 1024150204

Batch: 2023

Function Template

Write a function template to swap two variables of any data type.

```
main.cpp    Share  Run
```

```
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  void swapValues(T &a, T &b) {
5      T temp = a;
6      a = b;
7      b = temp;
8  }
9  int main() {
10     int x = 10, y = 20;
11     swapValues(x, y);
12     cout << "After swap: " << x << " " << y;
13     return 0;
14 }
```

```
Output 
```

```
After swap: 20 10

=== Code Execution Successful ===
```

Write a function template to find the minimum element in an array.

```
main.cpp    Share  Run
```

```
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  T findMin(T arr[], int n) {
5      T min = arr[0];
6      for(int i = 1; i < n; i++) {
7          if(arr[i] < min)
8              min = arr[i];
9      }
10     return min;
11 }
12 int main() {
13     int arr[] = {5, 2, 8, 1, 9};
14     cout << "Minimum: " << findMin(arr, 5);
15     return 0;
16 }
17
```

```
Output 
```

```
Minimum: 1

=== Code Execution Successful ===
```

Write a function template to sort an array using bubble sort.

```
main.cpp  [ ] [ ] [ ] Share Run

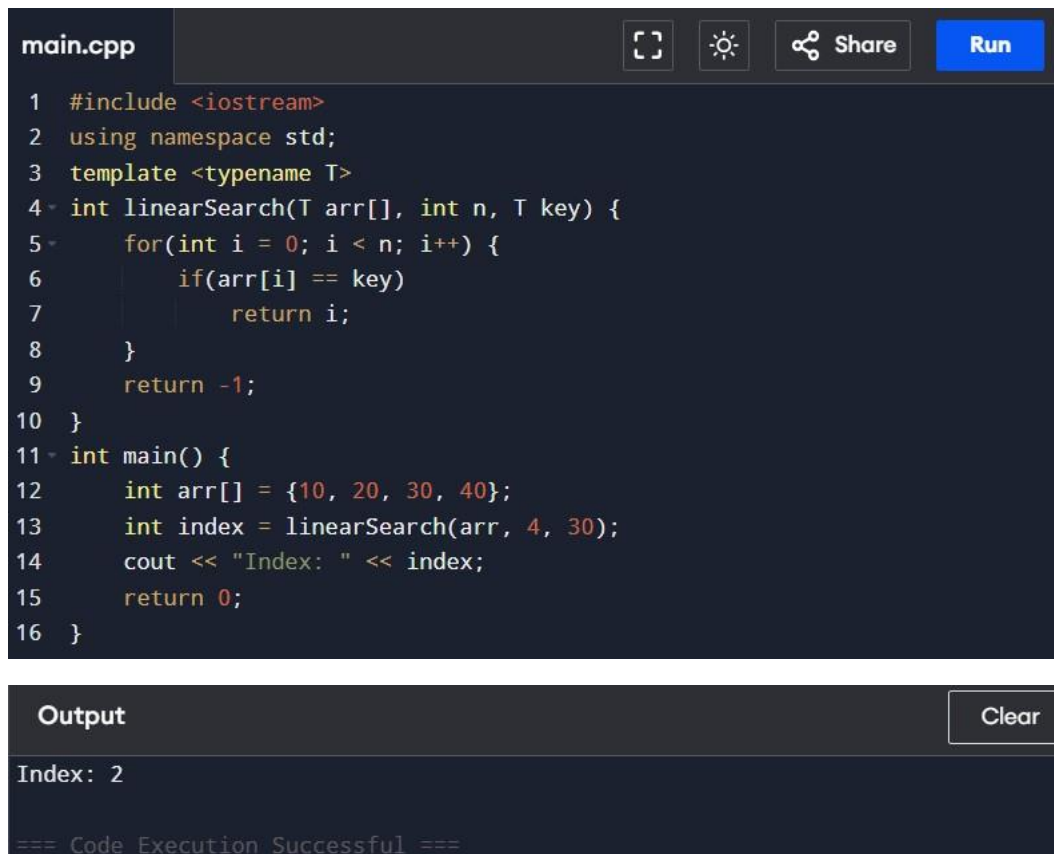
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  void bubbleSort(T arr[], int n) {
5      for(int i = 0; i < n-1; i++) {
6          for(int j = 0; j < n-i-1; j++) {
7              if(arr[j] > arr[j+1]) {
8                  T temp = arr[j];
9                  arr[j] = arr[j+1];
10                 arr[j+1] = temp;
11             }
12         }
13     }
14 }
15 int main() {
16     int arr[] = {5, 3, 8, 1};
17     bubbleSort(arr, 4);
18     for(int i = 0; i < 4; i++)
19         cout << arr[i] << " ";
20     return 0;
21 }
```

Output Clear

1 3 5 8

=== Code Execution Successful ===

Write a function template to perform linear search.



The screenshot shows a C++ IDE with a file named `main.cpp`. The code defines a function template `linearSearch` that takes an array `arr`, its size `n`, and a `key` to search for. It iterates through the array and returns the index of the first occurrence of the key, or `-1` if not found. The `main` function tests this with an array `{10, 20, 30, 40}` and a `key` of `30`, printing the result.

```
1 #include <iostream>
2 using namespace std;
3 template <typename T>
4 int linearSearch(T arr[], int n, T key) {
5     for(int i = 0; i < n; i++) {
6         if(arr[i] == key)
7             return i;
8     }
9     return -1;
10 }
11 int main() {
12     int arr[] = {10, 20, 30, 40};
13     int index = linearSearch(arr, 4, 30);
14     cout << "Index: " << index;
15     return 0;
16 }
```

The **Output** section shows the result of the program execution:

```
Index: 2
```

Below the output, it states: `=== Code Execution Successful ===`

Design overloaded versions of a function template `process()` such that:

- a) Accepts a single parameter
- b) Accepts two parameters of the same type
- c) Accepts two parameters of different types

main.cpp



Run

```
1 #include <iostream>
2 using namespace std;
3 // Single parameter
4 template <typename T>
5 void process(T a) {
6     cout << "Single param: " << a << endl;
7 }
8 // Two same type
9 template <typename T>
10 void process(T a, T b) {
11     cout << "Same type params: " << a << " " << b << endl;
12 }
13 // Two different types
14 template <typename T, typename U>
15 void process(T a, U b) {
16     cout << "Different types: " << a << " " << b << endl;
17 }
18 int main() {
19     process(10);
20     process(10, 20);
21     process(10, 2.5);
22     return 0;
23 }
```

Output

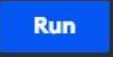
Clear

```
Single param: 10
Same type params: 10 20
Different types: 10 2.5

=== Code Execution Successful ===
```

Class Template

Develop a class template for stack with push and pop operations.

```
main.cpp    Share  Run
```

```
1 #include <iostream>
2 using namespace std;
3 template <typename T>
4 class Stack {
5     T arr[100];
6     int top;
7 public:
8     Stack() { top = -1; }
9     void push(T val) {
10         arr[++top] = val;
11     }
12     void pop() {
13         if(top >= 0)
14             top--;
15     }
16     void display() {
17         for(int i = 0; i <= top; i++)
18             cout << arr[i] << " ";
19     }
20 };
21 int main() {
22     Stack<int> s;
23     s.push(10);
24     s.push(20);
25     s.pop();
26     s.display();
27 }
```

Output 

10

=== Code Execution Successful ===

Develop a class template for queue with enqueue and dequeue operations.

main.cpp    Share 

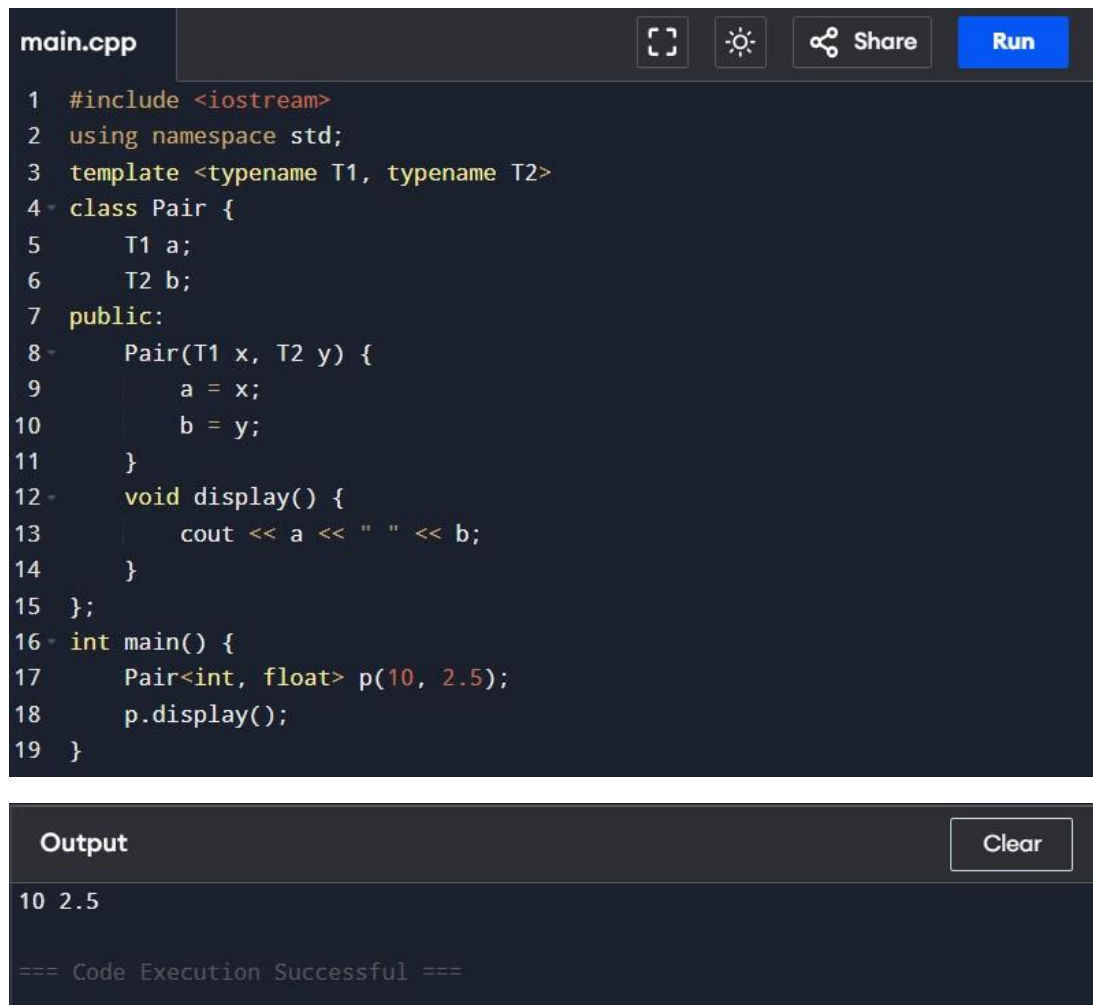
```
1 #include <iostream>
2 using namespace std;
3 template <typename T>
4 class Queue {
5     T arr[100];
6     int front, rear;
7 public:
8     Queue() { front = rear = -1; }
9     void enqueue(T val) {
10         if(front == -1) front = 0;
11         arr[++rear] = val;
12     }
13     void dequeue() {
14         if(front <= rear)
15             front++;
16     }
17     void display() {
18         for(int i = front; i <= rear; i++)
19             cout << arr[i] << " ";
20     }
21 };
22 int main() {
23     Queue<int> q;
24     q.enqueue(10);
25     q.enqueue(20);
26     q.dequeue();
27     q.display();
28 }
```

Output 

```
20

=== Code Execution Successful ===
```


Create a class template to store and display a pair of values.



The image shows a C++ IDE interface. At the top, there's a tab labeled 'main.cpp'. To the right of the tab are icons for a code editor (a square with a smaller square inside), a sun icon (likely for theme switching), a share icon, and a blue 'Run' button. The main area contains C++ code for a class template 'Pair'. The code is as follows:

```
1 #include <iostream>
2 using namespace std;
3 template <typename T1, typename T2>
4 class Pair {
5     T1 a;
6     T2 b;
7 public:
8     Pair(T1 x, T2 y) {
9         a = x;
10        b = y;
11    }
12    void display() {
13        cout << a << " " << b;
14    }
15 };
16 int main() {
17     Pair<int, float> p(10, 2.5);
18     p.display();
19 }
```

Below the code editor is an 'Output' panel. It has a 'Clear' button on the right. The output text is:

```
10 2.5
=== Code Execution Successful ===
```

3. Create a class template for basic arithmetic operations.

main.cpp

 Share

Run

```
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  class Arithmetic {
5      T a, b;
6  public:
7      Arithmetic(T x, T y) {
8          a = x;
9          b = y;
10     }
11     void add() { cout << "Add: " << a + b << endl; }
12     void sub() { cout << "Sub: " << a - b << endl; }
13     void mul() { cout << "Mul: " << a * b << endl; }
14     void div() { cout << "Div: " << a / b << endl; }
15 };
16 int main() {
17     Arithmetic<int> obj(10, 5);
18     obj.add();
19     obj.sub();
20     obj.mul();
21     obj.div();
22 }
```

Output

Clear

Add: 15
Sub: 5
Mul: 50
Div: 2

=== Code Execution Successful ===

Create a class template to input and display array elements.

The image shows a C++ IDE with a code editor and a console window. The code editor displays a C++ program that defines a template class `Array` with a static array `arr` of size 100. The class has two public methods: `input` and `display`. The `input` method takes a size parameter, sets `n` to that size, and reads `n` integers from standard input. The `display` method prints the `n` integers. The `main` function creates an `Array<int>` object, calls `input(3)`, and then `display()`.

```
1 #include <iostream>
2 using namespace std;
3 template <typename T>
4 class Array {
5     T arr[100];
6     int n;
7 public:
8     void input(int size) {
9         n = size;
10        for(int i = 0; i < n; i++)
11            cin >> arr[i];
12    }
13    void display() {
14        for(int i = 0; i < n; i++)
15            cout << arr[i] << " ";
16    }
17 };
18 int main() {
19     Array<int> obj;
20     obj.input(3);
21     obj.display();
22 }
```

The console window shows the input "1 2 3" and the output "1 2 3". It also displays the message "...Program finished with exit code 0" and "Press ENTER to exit console."